

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Parte 1 – Classi e Oggetti**

a cura di Massimo La Morgia*



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

** I crediti sulle slide di questo corso sono riportati nell'ultima slide

Letture

- oggetti (Horstmann capitolo 2.1–2.5)
- guida di stile Google per Java
<https://google.github.io/styleguide/javaguide.html>

Quali operazioni dovrete compiere per adattare questo programma python in Java?

```
FAHR = 40 # A constant in Python  
cel = (FAHR - 32) * 5.0/9.0  
print("The temp (in C) =", cel)
```

```
package lecture3;
/*
 * Convert Fahrenheit temperature to Celsius
 */
public class TempConvert {
    // The number we will convert
    public static final double FAHR = 40.0; // A
constant in Java
    public static void main(String[] args) {
        double cel; // inititalize
        cel = (FAHR - 32) * 5.0 / 9.0;
        System.out.println("The temp (in C) = " +
cel);
    }
}
```

Obiettivi

- ☐ classi vs. oggetti
- ☐ metodi, campi, overloading
- ☐ costruttori e la parola chiave new
- ☐ campi
- ☐ metodi statici vs. d'istanza
- ☐ incapsulamento
- ☐ uml
- ☐ stringhe
- ☐ heap, stack, metaspace
- ☐ input da terminale
- ☐ package

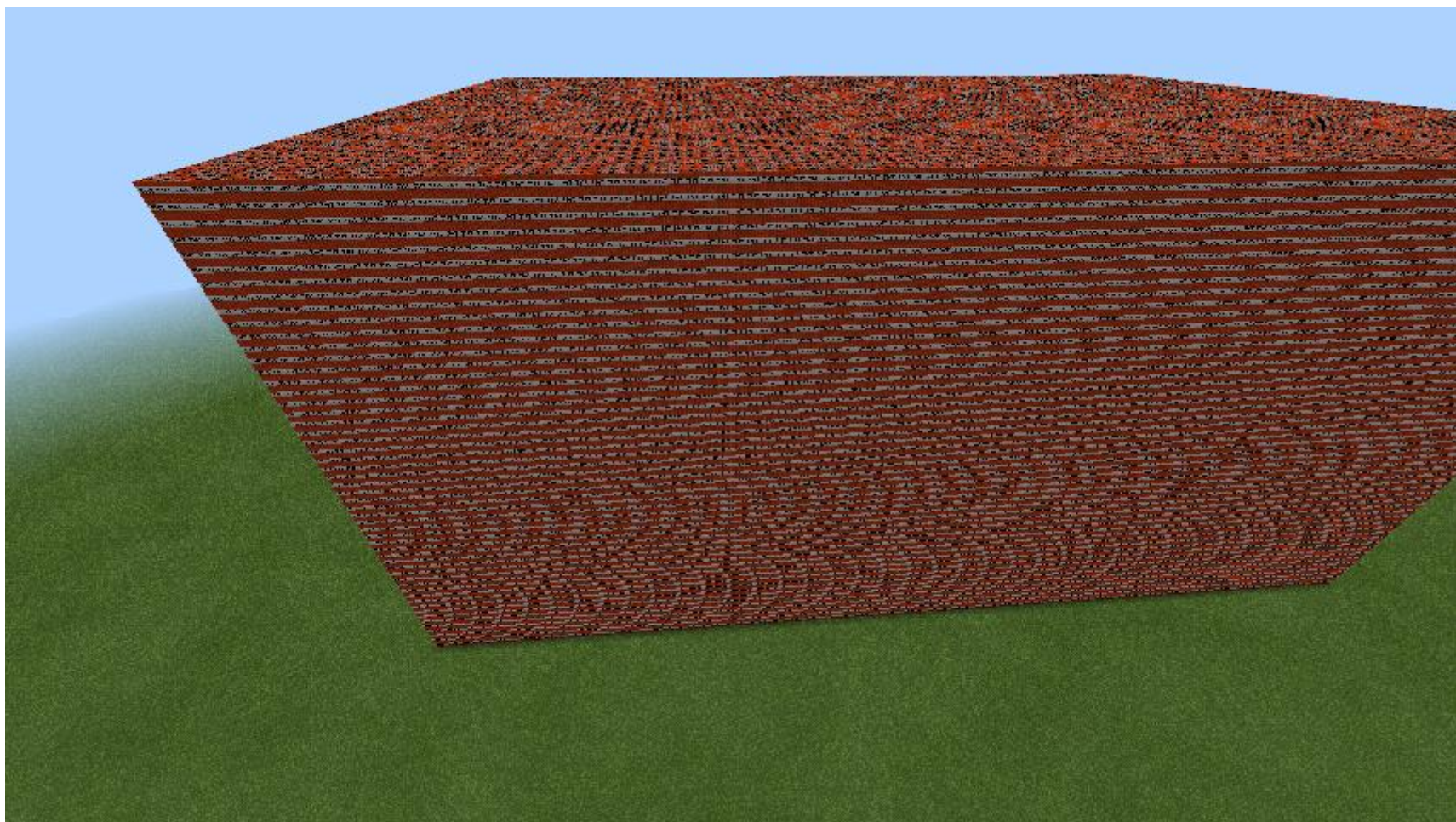




perché sono “macchine”?

Classe e Oggetto a confronto





Classe e Oggetto a confronto

- Acacia Fence - Acacia Fence Gate - Acacia Hanging Sign - Acacia Leaves - Acacia Log - Acacia Planks - Acacia Pressure Plate - Acacia Sapling - Acacia Sign - Acacia Slab - Acacia Stairs - Acacia Trapdoor - Acacia Wood - Activator Rail - Allum - Amethyst Cluster - Ancient Debris - Andesite - Andesite Slab - Andesite Stairs - Andesite Wall - Anvil - Azalea - Azalea Leaves - Azure Bluet - Bamboo - Bamboo Button - Bamboo Door - Bamboo Fence - Bamboo Fence Gate - Bamboo Hanging Sign - Bamboo Mosaic - Bamboo Mosaic Slab - Bamboo Mosaic Stairs - Bamboo Planks - Bamboo Pressure Plate - Bamboo Shoot - Bamboo Sign - Bamboo Slab	- Brick Wall - Bricks - Brown Banner - Brown Bed - Brown Candle - Brown Carpet - Brown Concrete - Brown Concrete Powder - Brown Glazed Terracotta - Brown Mushroom - Brown Mushroom Block - Brown Shulker Box - Brown Stained Glass - Brown Stained Glass Pane - Brown Terracotta - Brown Wool - Bubble Column - Bubble Coral - Bubble Coral Block - Bubble Coral Fan - Budding Amethyst - Cactus - Cake - Calotte - Calibrated Sculk Sensor - Campfire - Candle - Carrots - Cartography Table - Carved Pumpkin - Cauldron - Cave Vines - Chain - Chain Command Block - Chest - Cherry Button - Cherry Door - Cherry Fence - Cherry Fence Gate - Cherry Hanging Sign	- Cyan Banner - Cyan Bed - Cyan Candle - Cyan Carpet - Cyan Concrete - Cyan Concrete Powder - Cyan Shulker Box - Cyan Stained Glass - Cyan Stained Glass Pane - Cyan Terracotta - Cyan Wool - Damaged Anvil - Dandelion - Dark Oak Button - Dark Oak Door - Dark Oak Fence - Dark Oak Fence Gate - Dark Oak Hanging Sign - Dark Oak Leaves - Dark Oak Log - Dark Oak Planks - Dark Oak Pressure Plate - Dark Oak Sapling - Dark Oak Sign - Dark Oak Slab - Dark Oak Stairs - Dark Oak Trapdoor - Dark Oak Wood - Dark Prismarine - Dark Prismarine Slab - Dark Prismarine Stairs - Daylight Detector - Dead Brain Coral - Dead Brain Coral Block - Dead Brain Coral Fan - Dead Bubble Coral - Dead Bubble Coral Block - Dead Bubble Coral Fan - Dead Bush	- Gold Ore - Granite - Granite Slab - Granite Stairs - Granite Wall - Grass Block - Gravel - Gray Banner - Gray Bed - Gray Candle - Gray Carpet - Gray Concrete - Gray Concrete Powder - Gray Glazed Terracotta - Gray Shulker Box - Gray Stained Glass - Gray Stained Glass Pane - Gray Terracotta - Gray Wool - Green Banner - Green Bed - Green Candle - Green Carpet - Green Concrete - Green Concrete Powder - Green Glazed Terracotta - Green Shulker Box - Green Stained Glass - Green Stained Glass Pane - Green Terracotta - Green Wool - Grindstone - Hanging Roots - Hay Bale - Heavy Core - Heavy Weighted Pressure Plate - Honey Block - Honeycomb Block - Hopper - Horn Coral	- Lime Wool - Lodestone - Loom - Magenta Banner - Magenta Bed - Magenta Candle - Magenta Carpet - Magenta Concrete - Magenta Concrete Powder - Magenta Glazed Terracotta - Magenta Shulker Box - Magenta Stained Glass - Magenta Stained Glass Pane - Magenta Terracotta - Magenta Wool - Magma Block - Mangrove Button - Mangrove Door - Mangrove Fence - Mangrove Fence Gate - Mangrove Hanging Sign - Mangrove Leaves - Mangrove Log - Mangrove Pressure Plate - Mangrove Propagule - Mangrove Planks - Mangrove Roots - Mangrove Sign - Mangrove Slab - Mangrove Stairs - Mangrove Trapdoor - Mangrove Wood - Medium Amethyst Bud - Melon - Melon Stem - Monster Spawner - Moss Block - Moss Carpet - Mossy Cobblestone - Mossy Cobblestone Slab	- Pale Oak Planks - Pale Oak Pressure Plate - Pale Oak Log - Pale Oak Sapling - Pale Oak Sign - Pale Oak Slab - Pale Oak Stairs - Pale Oak Trapdoor - Pale Oak Wood - Pearlescent Froglight - Peony - Petrified Oak Slab - Piglin Head - Pink Banner - Pink Bed - Pink Candle - Pink Carpet - Pink Concrete - Pink Concrete Powder - Pink Petals - Pink Glazed Terracotta - Pink Shulker Box - Pink Stained Glass - Pink Stained Glass Pane - Pink Terracotta - Pink Tulip - Pitcher Crop - Pitcher Plant - Piston - Player Head - Podzol - Pointed Dripstone - Polished Andesite - Polished Andesite Slab - Polished Andesite Stairs - Polished Basalt - Polished Blackstone - Polished Blackstone Brick Slab	- Redstone Ore - Redstone Repeater - Redstone Torch - Redstone Wire - Reinforced Deepslate - Repeating Command Block - Respawn Anchor - Resin Bricks - Resin Brick Slab - Resin Brick Stairs - Resin Brick Wall - Resin Clump - Rooted Dirt - Rose Bush - Sand - Sandstone - Sandstone Slab - Sandstone Stairs - Sandstone Wall - Scaffolding - Sculk - Sculk Catalyst - Sculk Sensor - Sculk Shrieker - Sculk Vein - Sea Lantern - Sea Pickle - Seagrass - Short Grass - Shroomlight - Shulker Box - Skeleton Skull - Slime Block - Small Amethyst Bud - Small Dripleaf - Smithing Table - Smoker - Smooth Basalt - Smooth Quartz Block - Smooth Quartz Slab	- Torchflower Crop - Trapped Chest - Trail Spawner - Triprwire - Triprwire Hook - Tube Coral - Tube Coral Block - Tube Coral Fan - Tuff - Tuff Brick Slab - Tuff Brick Stairs - Tuff Brick Wall - Tuff Bricks - Tuff Slab - Tuff Stairs - Tuff Wall - Turtle Egg - Twisting Vines - Vault - Ventant Froglight - Vines - Warped Button - Warped Door - Warped Fence - Warped Fence Gate - Warped Fungus - Warped Hanging Sign - Warped Hyphae - Warped Nylum - Warped Planks - Warped Pressure Plate - Warped Roots - Warped Sign - Warped Slab - Warped Stairs - Warped Stem - Warped Trapdoor - Warped Wart Block - Water - Waxed Block of Copper
--	---	---	--	---	---	--	---

Classi e oggetti

Possono:

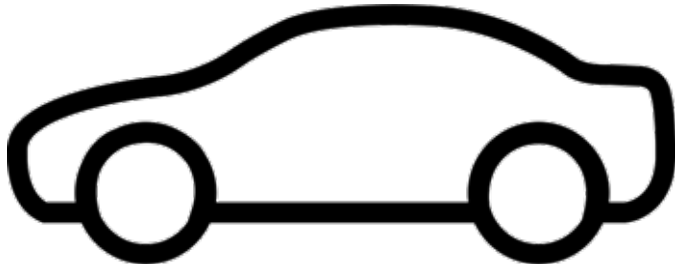
- Modellare gli oggetti del **mondo reale**
- Rappresentare oggetti **grafici**
- Rappresentare **entità software** (file, immagini, eventi, ecc.)
- Rappresentare **concetti astratti** (regole di un gioco, posizione di un giocatore)
- Rappresentare **stati** di un processo, di esecuzione, ecc.

Classi e oggetti

- Una **classe** è un pezzo del codice sorgente di un programma che **descrive** un particolare tipo di oggetti
- Le classi vengono definite dal programmatore (**definizione di classe**)
- La **classe** fornisce un **prototipo astratto** per gli oggetti di un particolare tipo
- Ne definisce la struttura in termini di:
 - **Campi** (stato) degli oggetti
 - **Metodi** (comportamenti) degli oggetti
- Un **oggetto** è un'**istanza** (un esemplare) di una **classe**
- Un **programma** può creare e usare **uno o più oggetti** (istanze) della **stessa classe**

Classe e Oggetto a confronto

Classe: automobile



- **Campi:**
 - **String** modello;
 - **Color** colore;
 - **int** numPasseggeri;
 - **double** benzina;
- **Metodi:**
 - Aggiungi/togli passeggero
 - Riempi serbatoio
 - Segnala quantità benzina

Definizione/Progettazione

Oggetto: una certa automobile



- **Campi:**
 - **String** modello = "5";
 - **Color** colore = Color.PINK;
 - **int** numPasseggeri = 1;
 - **double** benzina = 50;

• **Metodi:**
Come la classe

Tangibile/Implementazione

Classe e Oggetto a confronto

Classe:

- Definita mediante parte del codice sorgente del programma
- Scritta dal programmatore

Oggetto:

- Un'entità all'interno di un programma in esecuzione
- Creato quando un programma “gira” (dal metodo **main** o da un altro metodo)

a runtime

a runtime

Classe e Oggetto a confronto

Classe:

- Specifica la struttura (ovvero numero e tipi) dei campi dei suoi oggetti
- Specifica il comportamento dei suoi oggetti mediante il codice dei metodi

Oggetto:

- Contiene specifici valori dei campi; i valori possono cambiare durante l'esecuzione
a runtime
- Si comporta nel modo prescritto dalla classe quando il metodo corrispondente viene chiamato a tempo di esecuzione
a runtime

Classe e Oggetto a confronto

Classe: **Anello**



- **Campi:**

- `int x;`
- `int y;`
- `int r;`

- **Metodi:**

- `ruota`

Definizione/Progettazione

Oggetti:

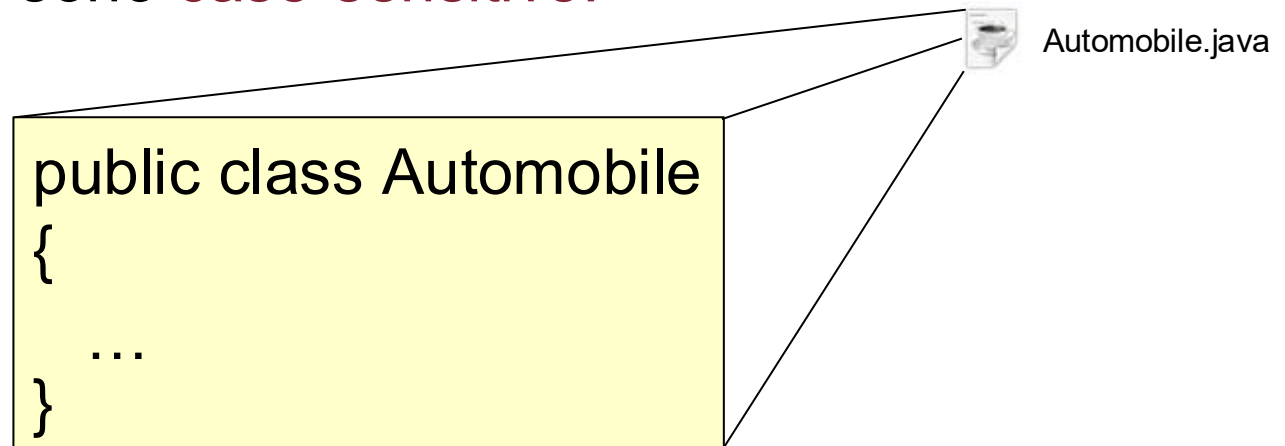
`anello1, ... , anello182`



a runtime

Classi e file sorgenti

- Ogni **classe** è memorizzata in un **file** separato
- Il **nome del file** DEVE essere lo **stesso** della **classe**, con estensione **.java**
- I nomi di classe iniziano sempre con una maiuscola (es. **Automobile**, non **automobile**)
- I nomi in Java sono **case-sensitive**!



Librerie

- I programmi Java normalmente non sono scritti **da zero**
- Esistono **migliaia di classi di libreria** per ogni esigenza (sia standard scritte da Sun/Oracle, sia sul Web scritte da centinaia di sviluppatori)
- Le **classi** sono organizzate in **package**
- Alcuni esempi:
 - **java.util** – classi di utilità
 - **java.awt** – classi per la grafica e le finestre
 - **javax.swing** e **javaafx** – sviluppo di interfacce GUI
- Un package “**speciale**” è **java.lang**: contiene le classi fondamentali per la programmazione in Java (es. **String**, **System**, ecc.)

Obiettivi

- ☒ classi vs. oggetti
- ☐ metodi, campi, overloading
- ☐ costruttori e la parola chiave new
- ☐ campi
- ☐ metodi statici vs. d'istanza
- ☐ incapsulamento
- ☐ uml
- ☐ stringhe
- ☐ heap, stack, metaspace
- ☐ input da terminale
- ☐ package

Come strutturare il codice di una classe in Java?



Esercizio: Un contatore

- Vogliamo realizzare una classe che rappresenta un contatore
- Il contatore permette di:
 - Incrementare il conteggio attuale
 - Ottenere il conteggio attuale
 - Resetare il conteggio a 0 (o a un altro valore)

Counter.java

```
public class Counter
```

```
{
```

Campi

```
/**  
 * Valore intero del contatore  
 */
```

```
private int value;
```

Commento Javadoc

Dichiarazione di un **campo**

Tipicamente i campi sono **privati**

Costruttore

```
/**  
 * Costruttore della classe  
 */
```

```
public Counter()
```

```
{
```

```
    value = 0;
```

```
}
```

Costruttore degli oggetti della classe

Inizializza il campo **value**

I **metodi** della classe sono **pubblici**

Metodi

```
/**  
 * Incrementa il contatore  
 */
```

```
public void count()
```

```
{
```

```
    value++;
```

```
}
```

Incrementa il valore di **value**

```
/**  
 * Ottiene il valore corrente del contatore  
 * @return valore intero del contatore  
 */
```

```
public int getValue() { return value; }
```

Restituisce il valore di **value**

```
}
```

Campi

- Un **campo** (detto anche **variabile di istanza**) costituisce la **memoria privata** di un oggetto
- Ogni **campo** ha un **tipo di dati** (es. il valore del contatore è intero)
- Ogni **campo** ha un **nome** fornito dal programmatore (es. value nella diapositiva precedente)

Dichiarare un campo

- La dichiarazione di un campo avviene come segue:

```
private [static] [final] tipo_di_dati nome;
```

Tipicamente ad **accesso privato**

Se specificato indica che il campo è **condiviso da tutti**

Se specificato indica che il campo è una **costante**

Es. **int**, **double**, **String**, ecc.

Esempi di dichiarazione

```
public class Hotel
{
    /**
     * Da evitare l'uso di una variabile "di comodo" come campo di una classe
     */
    private int k;

    /**
     * Nome dell'hotel
     */
    private String nome;

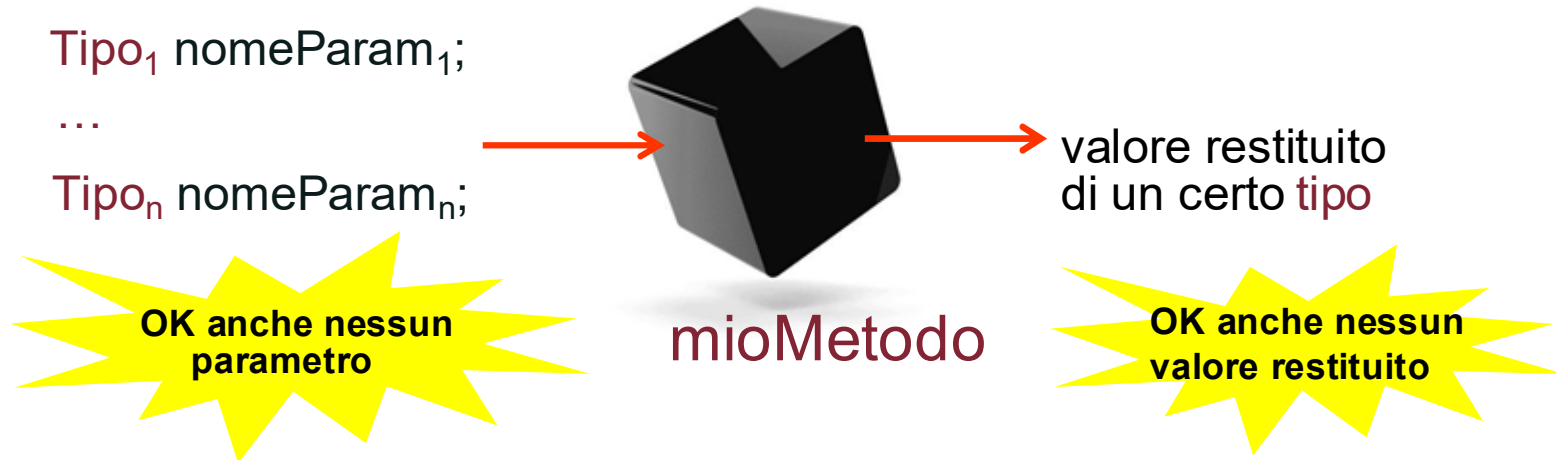
    /**
     * Numero di stanze
     */
    private int numeroStanze;

    /**
     * Superficie totale
     */
    private double superficie;

    /**
     * Sulla Guida Michelin
     */
    private boolean bGuidaMichelin;
}
```

Metodi

- Un **metodo** è tipicamente **pubblico**, ovvero visibile a tutti
- Il **nome di un metodo** per convenzione inizia con una lettera minuscola, mentre le parole seguenti iniziano con lettera maiuscola (es. `dimmiTuttoQuelloCheSai()`)
 - convenzione detta **CamelCase**



Definizione di un metodo

- La definizione di un metodo avviene come segue:

```
public tipo_di_dati nomeDelMetodo(tipo_di_dati nomeParam1, ..., tipo_di_dati nomeParamn)
```

```
{  
    istruzione 1;  
    .  
    .  
    istruzione m;  
}
```

Corpo del metodo
(racchiuso da parentesi graffe)

Nome del metodo

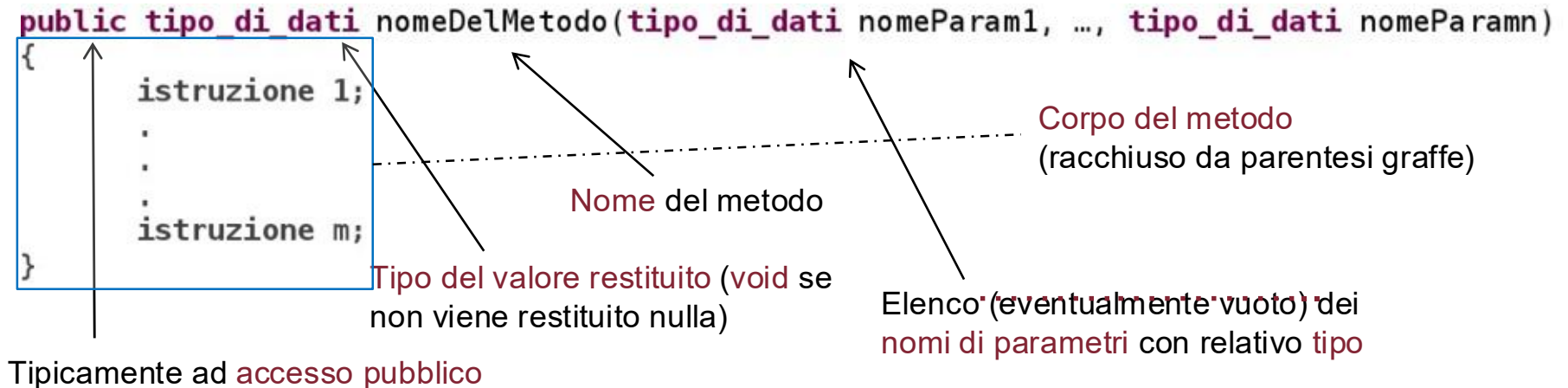
Tipo del valore restituito (void se
non viene restituito nulla)

Elenco (eventualmente vuoto) dei
nomi di parametri con relativo tipo

Tipicamente ad accesso pubblico

Definizione di un metodo

- La definizione di un metodo avviene come segue:



- Esempi:

```
public int getValue() { return value; }
public void reset(int newValue) { value = newValue; }
```

Metodi e valori restituiti

- Un metodo può **restituire** un valore al **chiamante**

```
public int getValue() { return value; }
```

- La parola chiave **void** nell'**intestazione** (“**signature**” o “**header**”) del metodo indica che il metodo non restituisce alcun valore:

```
public void reset(int newValue) { value = newValue; }
```

Costruttori

- **Metodi** (funzioni) per la **creazione** degli oggetti di una classe
- Possiedono **sempre** lo **stesso nome** della classe
- **Inizializzano** i **campi** dell'oggetto
- Possono prendere **zero, uno o più parametri**
- **NON** hanno **valori di uscita** (**MA** non specificano void)
- Una classe può avere anche **più costruttori** che differiscono nel numero e nei tipi dei parametri

Costruttori: esempio della classe Counter

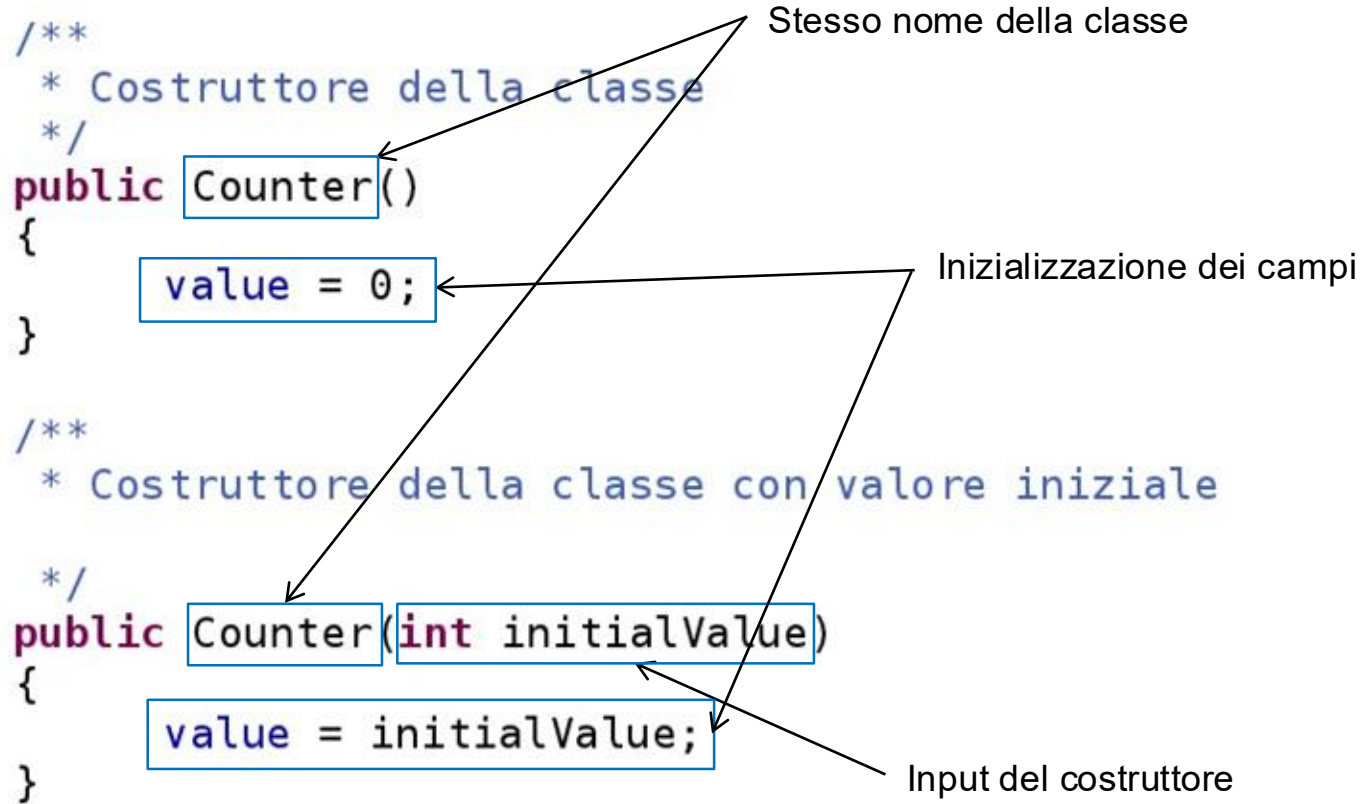
```
/**
 * Costruttore della classe
 */
public Counter()
{
    value = 0;
}

/**
 * Costruttore della classe con valore iniziale
 */
public Counter(int initialValue)
{
    value = initialValue;
}
```

Stesso nome della classe

Inizializzazione dei campi

Input del costruttore



Costruttori: creazione dell'oggetto

- Un oggetto viene creato con l'operatore **new** :

```
static public void main(String[] args)
{
    Counter contatore1 = new Counter();
    Counter contatore2 = new Counter(42);

    System.out.println("Valore del contatore1: "+contatore1.getValue());
    System.out.println("Valore del contatore2: "+contatore2.getValue());
}

/**
 * Costruttore della classe con valore iniziale
 */
public Counter(int initialValue)
{
    value = initialValue;
}
```

operatore new

Il numero, l'ordine e i tipi dei parametri **devono corrispondere**

Ancora sui costruttori

- Non è obbligatorio specificare un costruttore
- Se non ne viene specificato uno, Java crea per ogni classe un costruttore di default “vuoto” (senza parametri)
 - Inizializza le variabili d’istanza ai valori di default

Implementiamo il metodo reset

- **Versione 1:** semplicemente azzera il contatore

```
public void reset() { value = 0; }
```

- **Versione 2:** reimposta il contatore a un determinato valore

```
public void reset(int newValue) { value = newValue; }
```



Coding horror: campi pubblici

```
public class Punto
{
    public double x;
    public double y;
    public double z;

    public Punto (double x, double y, double z)
    {
    }

    public double getX()
    {
        return x;
    }

    public double getY()
    {
        return y;
    }

    public double getZ()
    {
        return z;
    }
}
```

Coding horror: il costruttore non salva i parametri

```
public class Segmento
{
    private Punto p1;
    private Punto p2;

    public Segmento(Punto p1, Punto p2)
    {

    }

    public static void main(String[] args)
    {
        Punto a = new Punto(1, 3, 8);
        Punto b = new Punto(4, 4, 7);
        Segmento s = new Segmento(a, b);
    }
}
```

Coding horror: inizializza due volte i campi

```
public class Segmento
{
    //Campi
    private Punto p1 = new Punto();
    private Punto p2 = new Punto();

    // Costruttore
    public Segmento(Punto pd1, Punto pd2)
    {
        p1=pd1;
        p2=pd2;
    }

    // Metodi

    // Main
    public static void main(String[] args)
    {
        Punto u1 = new Punto(1,3,8);
        Punto u2 = new Punto(4,4,7);
        Segmento s1 = new Segmento(u1,u2);
    }
}
```

Prima volta (inutile)

Seconda volta (corretto)

Coding horror: variabili di appoggio come campi!

```
public class MioArray
{
    int[] array;
    int varAppoggio = 0;
    int max = 0;
    int lunghezza = 0; /* Instance variables */
    int posMax;
    int[] arrayFirstLast = null;

    public MioArray(int[] array)
    {
        this.array = array;
    }
}
```

Chiamate di Metodi

- I metodi vengono chiamati su un particolare oggetto:

```
static public void main(String[] args)
{
    Counter contatore1 = new Counter();
    Counter contatore2 = new Counter(42);

    contatore1.count();
    contatore2.count();

    contatore2.reset();
    contatore1.reset(10);

    System.out.println("Valore del contatore1: "+contatore1.getValue());
    System.out.println("Valore del contatore2: "+contatore2.getValue());
}
```


Chiamate di Metodi

- Il **numero** e i **tipi** dei **parametri** (**argomenti**) passati a un metodo deve **coincidere** con i parametri formali del metodo

```
public void reset() { value = 0; }
public void reset(int newValue) { value = newValue; }
static public void main(String[] args)
{
    Counter contatore1 = new Counter();
    Counter contatore2 = new Counter(42);

    contatore1.count();
    contatore2.count();

    contatore2.reset();
    contatore1.reset(10);

    System.out.println("Valore del contatore1: "+contatore1.getValue());
    System.out.println("Valore del contatore2: "+contatore2.getValue());
}
```

Variabili locali vs. campi

- I **campi** sono variabili dell'oggetto
 - Sono visibili **almeno** all'interno di tutti gli oggetti della stessa classe
 - Esistono per tutta la vita di un oggetto
- Le **variabili locali** sono variabili definite all'interno di un metodo
 - Come parametri del metodo o all'interno del corpo del metodo
 - Esistono dal momento in cui sono definite fino al termine dell'esecuzione della chiamata al metodo in questione

Esercizio

- Progettare una classe **Persona** i cui oggetti rappresentano una persona e ne memorizzano il nome e il cognome
 - La classe espone un metodo **main** che crea un'istanza della Persona
 - La classe espone anche un metodo **stampa** che visualizza nome e cognome della persona

Esercizio

- Progettare una classe **Quadrato**, i cui oggetti sono costruiti con il lato dello stesso
 - La classe è dotata di un metodo **getPerimetro** che restituisce il perimetro
 - E di un metodo **main** che crea un quadrato di lato 4 e ne stampa a video il perimetro

Esercizio

- Progettare una classe **Cerchio** i cui oggetti rappresentano un cerchio
 - La classe è dotata dei metodi **getCirconferenza** e **getArea** (si usi la costante **Math.PI**)
 - La classe espone anche un metodo **main** che crea due cerchi (di raggio 1 e di raggio 5) e ne stampa la circonferenza (per il primo) e l'area (per il secondo)

Esercizio

- Progettare una classe **BarraDiCompletamento** i cui oggetti rappresentano una barra di caricamento
 - Gli oggetti vengono costruiti con la percentuale di partenza
 - La classe espone un metodo **incrementa** che, data una percentuale in input, incrementa la percentuale di partenza con quella fornita in input (ad es. **new**
 BarraDiCompletamento(5).incrementa(10) porta la barra al 15%)
 - Il metodo **toString** dell'oggetto restituisce una stringa contenente la percentuale di completamento arrotondata con **Math.round()**
 - Il metodo **main** che crea una barra di completamento che parte da 0, la incrementa prima di 20 punti percentuale e poi di altri 25 e quindi stampa la rappresentazione stringa della barra

Esercizio

- Progettare una classe **Rettangolo** i cui oggetti rappresentano un rettangolo e sono costruiti a partire dalle coordinate x, y e dalla lunghezza e altezza del rettangolo
- La classe implementa i seguenti metodi:
 - **trasla** che, dati in input due valori x e y, trasla le coordinate del rettangolo dei valori orizzontali e verticali corrispondenti
 - **toString** che restituisce una stringa del tipo “(x1, y1)->(x2, y2)” con i punti degli angoli in alto a sinistra e in basso a destra del rettangolo
- Implementare una classe di test **TestRettangolo** che verifichi il funzionamento della classe **Rettangolo** sul rettangolo in posizione (0, 0) e di lunghezza 20 e altezza 10, traslandolo di 10 verso destra e 5 in basso

Esercizio

Progettare una classe **Colore** i cui oggetti rappresentano un colore in modalità RGB e che sono costruiti a partire da tre valori: R (rosso), G (verde), B (blu), ognuno dei quali ammette un valore intero nell'intervallo 0-255.

- La classe Colore espone anche due costanti BIANCO e NERO
- Fare in modo che ogni Rettangolo (vedere esercizio precedente) abbia associato un Colore di base NERO e che sia possibile impostare il colore di un rettangolo mediante un apposito metodo

Tipi di dato in Java: valori primitivi vs. oggetti

- E' importante tenere a mente la differenza tra:
 - Valori di tipo primitivo (int, char, boolean, float, double, ecc.)
 - Oggetti (istanze delle classi)
- La loro rappresentazione in memoria è differente:
 - Valori primitivi: memoria allocata automaticamente a tempo di compilazione
 - Oggetti: memoria allocata durante l'esecuzione del programma (operatore new)

Inizializzazioni implicite per i campi della classe

- Al momento della creazione dell'oggetto i campi di una classe sono inizializzati automaticamente

Tipo del campo	Inizializzato implicitamente a
int, long	0, 0L
float, double	0.0f, 0.0
char	'\0'
boolean	false
classe X	null

- ATTENZIONE:** le inizializzazioni sono automatiche per i campi di una classe, ma **NON** per le variabili locali dei metodi

Ancora sulle notazioni dei letterali

- Potete dichiarare un intero in notazione decimale:

```
int val = 42;
```

- Un intero in esadecimale:

```
int val = 0x2A;
```

- Un intero in binario: `int val = 0b101010;`
- Il valore di quei 4 byte **sarà sempre lo stesso**: 42

Ancora sulle notazioni dei letterali

- Potete dichiarare un double in notazione decimale:

```
double val = 42.5;
```

- Un double in notazione scientifica:

```
double val = 0.425e2;
```

- Il float equivalente: `float val = 42.5f;`
- Il valore di quei 8 byte (o 4 byte, se float) **sarà sempre lo stesso: 42.5**

Esempio di inizializzazione implicita di campi

```
public class Room
{
    /**
     * Contiene il numero di persone attualmente nella stanza
     */
    private int numberOfPeople;

    /**
     * Conta il numero di accessi alla stanza
     */
    private Counter accessCounter;

    public static void main(String[] args)
    {
        int n;
        Counter myCounter;

        System.out.println(n+" "+myCounter);

        Room room = new Room();
        System.out.println(room.numberOfPeople);
        System.out.println(room.accessCounter);
    }
}
```

non inizializzato
esplicitamente

idem...

Errore a tempo di compilazione
**variable *n* may not have been
initialized**

Crea un nuovo oggetto in
memoria contenente i campi
numberOfPeople e
accessCounter **inizializzati**

Stampa **0**

Stampa **null**

Credits

Le slide di questo corso sono basate su quelle utilizzate dai Prof. Faralli e Samory, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.